

App Dev Project Report

1. Student Details

Name: Talbiya Parveen

Roll Number: 24f2001644

Email: 24f2001644@ds.study.iitm.ac.in

About Me: I am a Course Instructor at IIT Madras BS Degree program with a deep interest in web application development and data-driven technologies. I enjoy building meaningful applications that combine learning, analytics, and user experience.

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

Many students face difficulties in managing their placement-related activities, tracking applications, and keeping a record of interviews, offers, and deadlines. The Placement Portal Application aims to provide a **web-based platform** where users can log placement-related activities, track progress, visualize timelines, and analyze their placement preparation effectively. This helps students stay organized, monitor their progress, and improve their chances of securing placements.

Approach:

The Placement Portal Application is built using **Flask** for the backend with a modular file structure. Users can register, log placement-related activities (like applications sent, interviews scheduled, offers received), view progress dashboards and timelines, and analyze their placement preparation.

SQLAlchemy manages the SQLite database, while **CSS** and **Bootstrap 5** are used for interactive visualizations and responsive design.

3. AI/LLM Declaration

AI assistance was used in a limited capacity during the development of this project. Tools such as ChatGPT by OpenAI were used for:

- UI styling suggestions and improvements for HTML/CSS layouts
- Assistance in debugging errors during development

The extent of AI/LLM usage is around **10–15%**, limited to **code suggestions and documentation formatting**.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework used to build the web application
Flask-SQLAlchemy	ORM used to interact with the SQLite database
SQLAlchemy	Database abstraction layer used by Flask-SQLAlchemy
Flask-Login	Handles user login sessions and authentication
Flask-Bcrypt	Used for secure password hashing
Flask-Migrate	Handles database migrations using Alembic
Alembic	Tracks database schema changes
Jinja2	Renders dynamic HTML templates
Werkzeug	Provides routing, request handling, and development server features for Flask

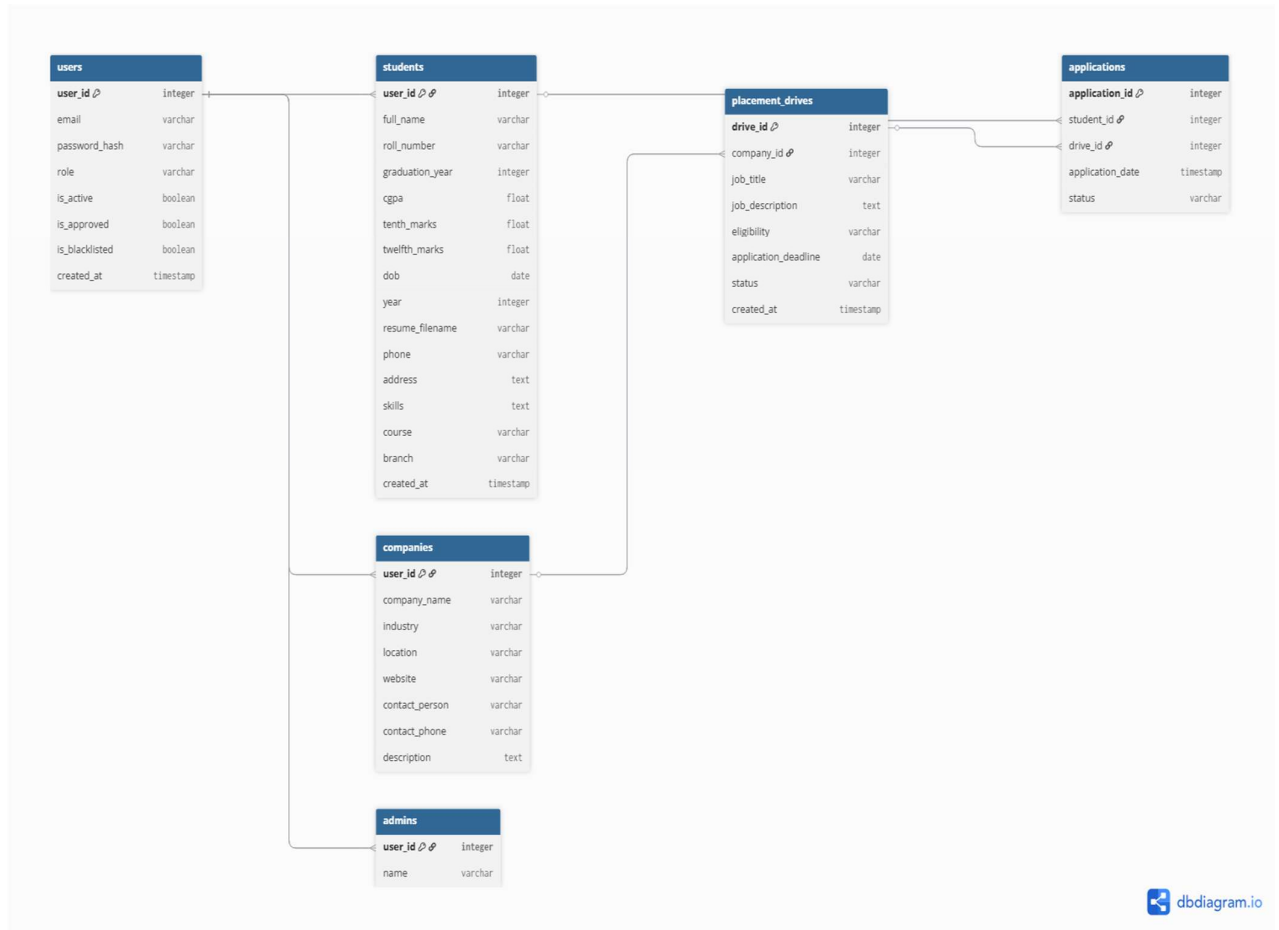
5. Database Schema / ER Diagram

Tables:

- **User** — stores user account details
(*user_id, email, password_hash, role, is_active, is_approved, is_blacklisted, created_at*)
- **Student** — stores student profile information
(*user_id, full_name, roll_number, graduation_year, cgpa, tenth_marks, twelfth_marks, dob, year, resume_filename, phone, address, skills, course, branch, created_at*)
- **Company** — stores company profile details
(*user_id, company_name, industry, location, website, contact_person, contact_phone, description*)
- **Admin** — stores administrator details
(*user_id, name*)
- **PlacementDrive** — stores job drive information created by companies
(*drive_id, company_id, job_title, job_description, eligibility, application_deadline, status, created_at*)
- **Application** — stores student applications for placement drives
(*application_id, student_id, drive_id, application_date, status*)

Relationships:

- One-to-One → User → Student
- One-to-One → User → Company
- One-to-One → User → Admin
- One-to-Many → Company → Placement Drive
- One-to-Many → Student → Application
- One-to-Many → Placement Drive → Application



6. API Resource Endpoints

Endpoint	Method	Description
/login	POST	Authenticate user
/student/dashboard	GET	Student dashboard
/company/dashboard	GET	Company dashboard
/admin/dashboard	GET	Admin dashboard

/company/create_drive	POST	Company creates placement drive
/student/apply/<drive_id>	POST	Student applies for drive

YAML API Definition File:

Included separately in the submission ZIP as api.yaml.

7. Architecture and Features (optional)

Architecture Overview:

- **run.py** – main Flask application entry point
- **auth/** – Handles login, logout, and authentication routes.
- **admin/** – Contains routes for admin dashboard and management.
- **student/** – Includes student dashboard and application features.
- **company/** – Handles company dashboard and drive creation.
- **templates/** – Stores HTML templates for the frontend.
- **static/** – Contains CSS, JavaScript, and other static files.
- **models.py** – Defines database models using SQLAlchemy.
- **utils.py** – Contains helper functions such as role-based access control.
- **init.py** – Initializes the Flask app and registers blueprints.

Features:

- User registration and login
- Role-based authentication (Student / Company / Admin)
- Student dashboard to view and apply for placement drives
- Company dashboard to create and manage job drives
- Admin dashboard to monitor system users
- Secure password hashing
- Dynamic page rendering using Jinja2 templates
- Modular backend structure using Flask Blueprints

Additional Features:

- Organized template folder structure for scalability
- Clean separation of authentication and role-based modules
- Deployment-ready project structure

8. Video Presentation

Drive Link:

<https://drive.google.com/file/d/1NeVTbRg2Q2-OgyiYLyivLQ1FE4Lt-bOE/view?usp=sharing>
